

Baseline Planning Suite

Senior Frontend Engineer — case study

Baseline answers one question for a delivery organisation: who is working on what, for how long, and what it costs. Two teams ship it independently and never import each other's source.

STACK	React 18+, TypeScript strict
DELIVERABLE	Three federated builds — shell, people, delivery
ONE COMMAND	<code>docker compose up</code> → localhost:8080

What you're building

A delivery organisation sells its people's time. Someone has to say, month by month, who is assigned to which piece of work, whether anyone is committed beyond their contracted hours, and what the plan costs at the rates those people are paid. Baseline is where that plan is made and priced.

Two facts make it awkward. A person's cost rate changes over time, so one month can be priced at two rates. And the same people are shared across projects, so capacity only means anything when every project is counted together.

The register of people and their rates, and the plan that spends them, are owned by different teams on different release schedules. That is why this is a micro-frontend exercise and not one application: three separate apps, built by two teams that ship on their own schedules.

People the employee register: roles, weekly hours, and cost-rate history.

Delivery a work breakdown and a month-by-month staffing grid that spends the rates People owns.

Shell hosts both; owns navigation, display currency and the active user.

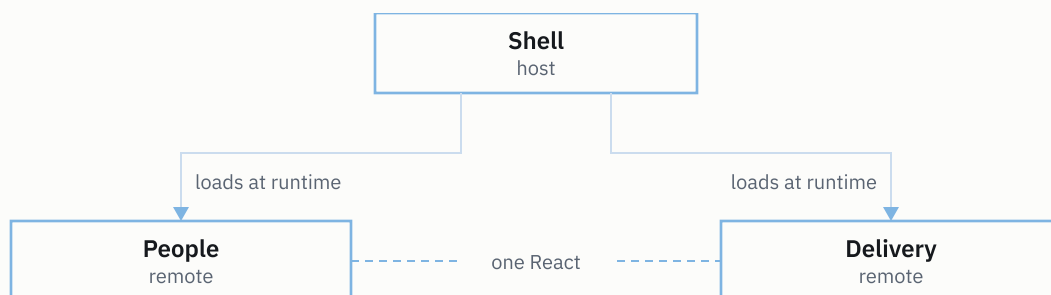


FIGURE 1

Topology. The dashed line is the shared React singleton.

How we assess it

Four dimensions, weighted. Spend your time where the weight is; the reference calculation in 3.3 is the first thing we check.

DIMENSION	WEIGHT	WHAT EARNS IT
Architecture and boundaries	35	Where domain logic lives and whether it is testable without a browser; whether the two remotes are genuinely independent or quietly coupled.
Domain correctness	30	Rate splitting, working-day arithmetic, unit conversion, roll-ups, totals that reconcile. We have specific inputs we will type in.
Micro-frontend engineering	20	Runtime remote resolution, the singleton story, standalone and hosted from one build, isolation on failure.
Code quality	15	Types that make wrong states unrepresentable, domain naming, consistency across the three apps, no dead scaffolding.

NOT SCORED Visual polish, a design system, auth, mobile, offline, scheduling.

Requirements

3.1 SCOPE

AREA	WHAT HAS TO WORK
Shell	Navigation between the remotes; owns display currency and the active user and pushes both in at runtime.
People	Searchable register; open an employee and edit their rate history — rates addable, correctable and removable, including retroactively.
Delivery	A work breakdown tree with create, rename, move, delete; a staffing grid of people × months, every leaf cell editable.
Across both	A rate edited in People reaches any open Delivery cost view with no reload. Over-capacity is flagged in both apps.
Resilience	If a remote fails to load, the shell stays alive and says so in place of that panel. Give us a way to trigger it.

3.2 DATA MODEL

Minimum fields. Name them how you like, add what you need, but every one of these has to exist somewhere.

ENTITY	FIELDS	NOTES
Employee	id, name, role, weeklyHours	weeklyHours is 40, 32 or 20
RateRecord	id, employeeId, validFrom, hourlyCost	No end date — a record runs until the next one starts
Project	id, name, startDate, endDate	Projects overlap and share people
BreakdownItem	id, projectId, parentId, name	parentId null at the root; three levels deep
Allocation	id, breakdownItemId, employeeId, month, amount	One canonical unit only — you choose which

FIGURE 2 Allocation.amount is stored in a single unit; the four display units in R2 are conversions of it, never separate fields.

FIXTURES A seed file with fixed IDs ships with the exercise; A. Okafor and the 12 March 2026 rate change used in the reference calculation are in it. It holds 60 employees, 150 rate records (one to four per person, 10 of them changing mid-month), 4 overlapping projects sharing people, 90 breakdown items three levels deep, and a twelve-month horizon of 720 editable cells. Those are the file's counts, not targets for you to generate.

3.3 DOMAIN RULES

Rates are effective-dated, and months split. A rate applies from its validFrom until the next one begins; the last rate has no end, and validFrom is inclusive — the day itself is priced at the new rate. An allocation is spread evenly across the working days of its month, so every working day carries the same effort. That month's cost is then the working days before the change at the old rate plus the working days from the change onward at the new rate; more than one change in a month simply yields more than two slices. Working days are Monday to Friday — public holidays are ignored entirely. An allocation in a month earlier than the employee's first rate record costs zero, and the cell is marked.

R1

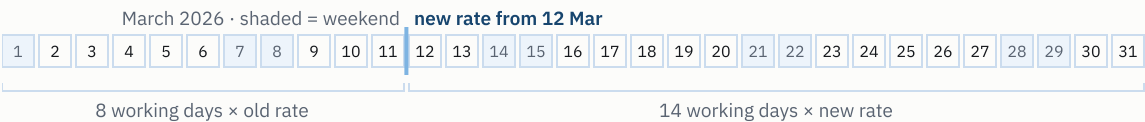


FIGURE 3 One month, one mid-month rate change: two slices, each priced at its own rate.

Four units, one truth. The grid reads and edits in hours, person-months, % of capacity, or cost. R2
Store one canonical unit and convert only at the edges. One person-month = weekly hours × (working days that month ÷ 5), so it varies by person and by month — it is never a constant. % of capacity is percent of that person's person-month for that month: 100% is exactly one person-month. Editing a cell in € in a month that contains a rate change divides the amount entered by that cell's blended rate for the month — €89.5455/h in the reference calculation below — giving hours, which convert to the canonical unit. Display precision is fixed: hours 2dp, person-months 2dp, % 1dp, cost 2dp. Switching units and switching back must not change the stored value.

INPUTS A. Okafor, 40 h/week. Rates €80.00/h from 2025-01-01 and €95.00/h from 2026-03-12. One leaf cell of 0.50 person-months in March 2026.

March 2026 working days	22
Working days before 12 Mar	8 validFrom is inclusive
Working days from 12 Mar on	14
One person-month	$40 \times 22 \div 5 = 176.00 \text{ h}$
This allocation in hours	$0.50 \times 176 = 88.00 \text{ h}$
Hours per working day	$88 \div 22 = 4.00 \text{ h}$
Cost	$8 \times 4 \times 80 + 14 \times 4 \times 95 = 2,560 + 5,320 = €7,880.00$
Same cell in % of capacity	50.0%
Implied blended rate	€89.5455/h

FIGURE 4
Reference calculation. If your build does not produce these five numbers, stop and fix that before anything else.

PM	Hours	%	€					Apr 26 – Mar 27 · 60 people			
WORK PACKAGE / PERSON				APR 26	MAY 26	JUN 26	JUL 26	AUG 26	SEP 26	TOTAL	
Ledger migration DERIVED				2.10	2.85	3.05	2.40	1.75	0.90	13.05	
L. Okafor				0.80	1.00	1.15	0.90	0.60	0.25	4.70	
M. Brandt				0.70	0.95	1.18 †	0.80	0.65	0.40	4.68	
S. Haddad				0.60	0.90	0.72	0.70	0.50	0.25	3.67	
Reporting cut-over DERIVED				0.00	0.45	1.60	2.20	2.05	1.10	7.40	

M. Brandt is over capacity in Jun 26 once his other projects are counted.

FIGURE 5
Person-months. Outlined cell is mid-edit; † is over capacity; derived rows roll up from their children.

Totals must add up. Totals are computed from exact values and rounded only for display. The displayed total must equal the sum of the displayed cells: use largest-remainder distribution so the rounded cells add to the rounded total exactly. The 0.01 tolerance is a floating-point allowance, not a rounding budget. R3

Parents are derived. Effort and cost on a parent come from its children and are read-only. You decide what happens to a leaf's own allocation when a child is added beneath it; two resolutions are acceptable — move the leaf's allocation onto the new child, or refuse the insertion with a message. Both are fine; silent loss is not. R4

Capacity is cross-project. Capacity for a month is 100% of that person's person-month, and allocation is summed across every project — including ones not currently open. When that sum exceeds capacity, People shows the person as oversubscribed and Delivery names the assignment that caused it, meaning the most recently edited allocation contributing to that person-month. The edit is flagged, never blocked.

R5

Hard constraints

4

Delivery prices its grid using rates that People owns. Whether Delivery reads rate records and computes cost itself, or asks People for a computed cost, is the decision we are assessing. Either can be right — make the choice deliberately and defend it in your README.

GIVEN

- **No UI libraries** — no component kit, headless primitives, table, grid or tree package. Styling tooling and date libraries are fine.
- **Three federated builds** — shell, people and delivery, wired with Module Federation.
- **Remote URLs resolve at runtime** from container configuration, never from the bundle.
- **Standalone and hosted** — each remote runs both ways from one codebase and one build.
- **One command** — `docker compose up` from a clean clone serves the suite on `localhost:8080`; no Node on the host.
- **TypeScript strict** — no `any`.

YOURS

- **The data layer** — how each service gets its data and where it lives.
- **Transport between the remotes** — how a change in one reaches the other.
- **State ownership** — one owner per piece of data, a published contract, never internals.
- **The persistence mechanism** — which store, given that edits must survive a reload.
- **The bundler.**

Handover

5

The repository, with real commit history.

`README.md` — how to run it, how to break a remote on purpose, a map of the repo.

Tests where you would defend them; we look hardest at calculation logic that runs without mounting React.

Then we walk the code together and make a small change in it, live.